

ML System Design Document

Human, Hand, and Gesture Tracking on-the-edge

1. Problem Framing & Requirements

Restate your Industry 4.0 problem clearly

Our Industry 4.0 problem is how to build a practical edge vision system that can recognize when a person is near a machine and tell whether that person is giving a simple hand command. This is important for safe human-machine interaction especially in Industry 4.0 situations like those using co-operator robots (cobots). In many industrial settings, people and robots work close together, so a system that can understand body position and a few clear signals such as stop or go could improve awareness and add another layer of support around machine operation. We explored HOG as an early direction for person detection, but we determined it was not the right long-term fit because it mainly gives rough person detection and does not naturally support the body landmarks, wrist cues, and gesture reasoning that this project needs. For that reason, the proposed system is built around MediaPipe Tasks instead of HOG.

Define success metrics: How will you measure if your system works? (e.g., accuracy targets, cost reduction, defect detection rate)

We will measure success in a few simple ways. First, the system should reliably detect when a person is present in the camera view. Second, the pose landmarks should be stable enough that the wrist locations are usable for a) locating hands more easily and b) picking likely hand signals/gestures. Third, the signal/gesture should correctly identify a small command like an open hand/palm for STOP and thumbs-up for GO. We would measure this with practical measures like false alarm rate and missed command rate. We would also track end-to-end latency and approximate frame rate. The goal of these measurements is to ensure a system that is accurate but not too slow (which would not be very useful near live equipment).

Identify constraints: real-time processing needs, computational resources, acceptable latency, budget limitations

The main constraints are real-time performance, limited compute, and low-cost hardware. The design is meant to run on an Intel NUC (NUC5i3MYHE), so we need lightweight models to not

bog it down too much. Acceptable latency should be low enough that a person can make a simple hand signal and get a useful system response quickly, rather than several seconds later. Lighting changes, clutter, occlusion, and different worker positions are also practical constraints. Budget matters too, so the design should stay realistic for local deployment. To do this we're using a standard webcam and an edge computer instead of a more expensive industrial vision stack.

2. Data Pipeline Design

Data Sources: What data will you collect and from where? (sensors, cameras, databases, manual entry)

The main data source will be a USB webcam connected to the Intel NUC. That camera will provide the live video frames used by the system. We may also keep simple local event logs generated by the vision pipeline and manually labeled gesture examples for evaluation and future tuning.

Data Collection: How frequently? Batch or real-time streaming? What format?

The main system will use real-time streaming because the goal is to respond to live human presence and hand signals near equipment as they happen. The webcam will continuously produce image frames, and those frames will be processed as they arrive. Instead of saving full video all the time, the system will initially collect lightweight structured data in the form of CSV event logs and debug log files. These logs can record items such as timestamp, frame number, whether a person was detected, whether a wrist-centered ROI was found, the predicted gesture, confidence score, and the mapped command such as STOP or GO. To support troubleshooting and historical review without overloading the system, the pipeline may also save snapshots only at important moments, such as when a gesture is detected or when confidence passes a threshold. The exact snapshot filename or file path can then be written into the same CSV row so each logged event can be tied back to a specific saved image. This approach keeps the system lightweight while still creating useful records for later evaluation, debugging, and future model improvement. The raw input format is live video frames from the webcam, while the stored data format will primarily be CSV logs, text-based debug logs, and selected image snapshot files.

Data Storage: Where will data be stored? (local databases, cloud storage, data lake)

For this project, the simplest and most realistic storage approach is local storage on the Intel NUC. We plan to store event logs locally in a CSV file. Each entry could include the timestamp,

whether a person was detected, the predicted gesture, the confidence if available, and a note about whether an image or short clip was saved. We may also save small image snapshots or short video clips for debugging and evaluation.

Data Preprocessing: What cleaning, normalization, or feature engineering steps are needed?

The preprocessing should stay simple. We would make sure the pose model would provide body landmarks, especially the wrist points. Those wrist landmarks would then be used to choose likely hand regions for the gesture stage. In this design, the most important preprocessing step is really the wrist-guided hand selection, which narrows the gesture task to the most relevant part of the frame.

Data Labeling: How will you obtain labeled training data? (historical records, manual labeling, semi-automated)

We would mainly use manual labeling at first. We can record examples of people performing the target gestures, then label those samples as STOP, GO, or no valid command. As the project grows, maybe we could use a semi-automated approach where the system flags likely gesture events and we review them instead of labeling every frame from scratch.

3. Model Selection & Training

Which ML algorithm(s) will you use and why? (reference techniques from Weeks 1-9)

We plan to use MediaPipe Pose Landmarker Lite, MediaPipe Hand Landmarker, and MediaPipe Gesture Recognizer. In the simplified final pipeline, the main logic is MediaPipe Pose plus a wrist-guided Gesture Recognizer. We explored HOG because it is a classic computer vision baseline and it helped us think through person detection, but we determined it was not the right direction for the full project. HOG is more limited and is not naturally set up for landmark-based body reasoning or gesture recognition (it was also pretty low accuracy). The MediaPipe task models are a better fit because they already support body landmarks, hand-focused analysis, and gesture classification in a lightweight on-device workflow (big time-saver). To reference Weeks 1-9, these models are based on the CNN which we learned about.

What features will you input to the model?

The system input is mainly live image frames from the webcam. At the system level, the important information passed from one stage to the next is the pose output, especially the wrist

landmarks, and then the wrist-centered hand regions sent into the gesture stage. We are not manually designing low-level features like gradient histograms. Instead, we are using model-generated landmarks and focused image regions as the practical features that drive later decisions.

How will you split data for training/validation/testing?

If we build a project dataset of gesture examples, we would split it into training, validation, and testing sets. A simple plan would be 70% for training, 15% for validation, and 15% for testing. We would want samples from different distances, lighting conditions, body positions, and backgrounds in each split so the evaluation is not overly easy. If the dataset stays small, we could also use repeated validation or cross-validation style testing to get a more stable view of performance.

What evaluation metrics will you use? (accuracy, precision, recall, RMSE, etc.)

We would mainly use accuracy, precision, recall, and confusion matrices for gesture recognition. Precision matters because too many false commands would make the system annoying or unsafe. Recall matters because missed commands, especially missed STOP signals, would also be a problem. We would also track practical deployment metrics such as frame rate and latency. RMSE is not really relevant here because this is mainly a detection and classification problem, not a regression problem.

How will you handle class imbalance or data quality issues?

We would handle class imbalance by collecting a balanced set of examples for each command gesture and also collecting negative examples where no valid command is present. For data quality, we would collect samples under different lighting, angles, and distances, and we would review failure cases to improve the dataset. If a particular light source, angle, or background causes trouble, we would add more examples of that case instead of assuming the model will generalize automatically.

4. Deployment & Integration

Where will the model run? (edge devices, local servers, cloud)

The model will run on an edge device, specifically an Intel NUC running Ubuntu. That is one of the main design goals of the project. We want local edge-computing so the system does not depend on cloud connectivity and can stay responsive near the machine even when the internet goes out.

How will it integrate with existing systems? (connects to manufacturing execution system, alerts operators, automatic shutdowns)

The simplest integration is a local operator display that shows detected people and recognized gestures in real time. Beyond that, the system could write events to a local log and send simple alerts to another local application or control interface. In a more advanced version, we could have an audio alert or a voice repeating the command “STOP/GO” so that operators could hear the model’s prediction rather than visually looking for it.

What happens when the model makes a prediction? (flag for review, automatic action, operator notification)

When the model makes a prediction, the first step is to display and log it. If a person is detected, the system can mark the area as occupied. If an Open_Palm gesture is detected, the system can log and display a STOP event. If a Thumb_Up gesture is detected, the system can log and display a GO event. In the early version of the system, those predictions should be advisory rather than directly controlling machinery. That means they can notify an operator or be flagged for review, but they should not replace standard industrial safety controls.

Real-time or batch predictions?

The system is intended for real-time predictions. The purpose of the project is to recognize live human presence and commands near equipment, so real-time inference is the correct operating mode.

5. Monitoring & Maintenance

How will you monitor model performance over time?

We would monitor both technical performance and practical behavior. On the technical side, we would track frame rate, latency, and how often the system loses a person or hand cue. We would review the local event log and saved examples to see how often STOP and GO are predicted correctly and how often false commands happen. Over time, this would show whether the system is staying reliable in the real-world environment.

What triggers model retraining? (performance degradation, new data availability, seasonal changes)

Retraining or model adjustment would be triggered by visible performance degradation, such as more false positives, more missed gestures, or weaker wrist guidance. It could also be triggered by new data availability, especially if we collect more examples from the real target environment. Changes in lighting, camera placement, or workspace layout/clutter could also justify updating the model.

Who is responsible for maintaining the system?

For this project, we as the project team would be responsible for maintaining the system. In a real industrial setting, that job would likely belong to the engineering team responsible for the local vision or automation setup.

How will you handle model updates without disrupting operations?

We could handle updates in a “sandbox” by testing them offline first on saved clips and logged examples before replacing the active version. That way, we can compare the new version to the old one without interrupting normal operation.

6. Challenges & Risk Mitigation

What are the biggest risks or challenges for this system? (data availability, computational costs, user adoption, safety concerns)

The biggest risks are limited compute on the Intel NUC, environmental variability, user trust, and safety. Since we are running on low-end hardware there will be a tradeoff between speed and robustness. Lighting changes, clutter, and unusual body positions could hurt performance. If the system gives too many false signals, operators may stop trusting it. Of course, safety is the biggest concern/risk.

What's your backup plan if the ML model fails or performs poorly?

Our backup plan is to reduce the system's responsibility. If the model starts to perform poorly, we could simplify the pipeline and keep only person or body awareness. If the system fails we'd

revert to true machine constraints and ensure the system movements are restricted/slowed and indicate safe areas for humans and restricted zones humans will need to avoid.

Are there any ethical, safety, or regulatory considerations?

Yes. Safety is the biggest one. The model should not replace standard industrial safety controls, emergency stops, or lockout procedures. It should act as an added intelligence layer, not the only protection. There are also ethical and privacy considerations. Workers should know that the system is observing gestures and body position, and we should avoid storing more personal imagery than needed. Logs should focus on events and system performance rather than identity. If the project ever moved into a real industrial deployment, data retention, privacy expectations, and plant safety requirements would all need to be reviewed carefully.